

Assignment 1

Builder Design Pattern

Custom Car Builder

Name: Marat Ulan

Group: SE-2532

Submission date: 12.09.2026

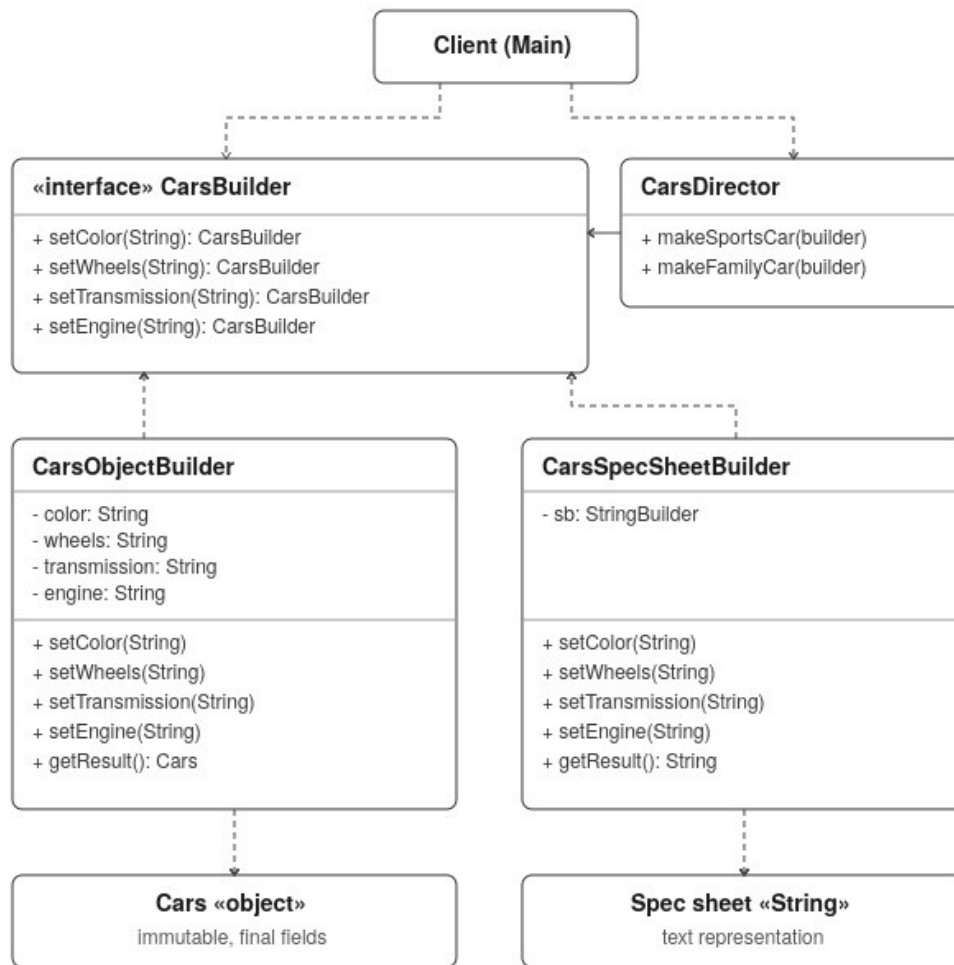
1. Introduction

This report documents the implementation of the Builder design pattern applied to a custom car configuration system. The Builder pattern separates the construction of a complex object from its representation, allowing the same step-by-step construction process to produce different outputs. In this project, the same sequence of construction steps — setting the color, wheels, transmission, and engine — is reused to produce two different representations of a car: a strongly-typed, immutable Cars object, and a human-readable text spec sheet (similar to a dealership configuration printout).

The goal of the assignment was to design a fluent Builder API, provide a Director that encapsulates ready-made car configurations, keep the product immutable, validate input before the object is built, and avoid magic numbers/strings and oversized constructors. The sections below present the class structure (UML diagram), a discussion of five Clean Code principles applied in the implementation with real code excerpts, and a conclusion.

2. Structure — UML Class Diagram

The diagram below shows the classes of the system and their relationships: the CarsBuilder interface, its two concrete builders (CarsObjectBuilder and CarsSpecSheetBuilder), the immutable Cars product, the CarsDirector, and the Main client. Note that getResult() is declared on each concrete builder rather than in the interface, because the two builders return different result types (Cars vs. String). The Director depends only on the shared CarsBuilder interface, never on a concrete builder, so it can direct either representation interchangeably.



Two builders produce two representations from the same construction steps.
 Dashed = implements / creates / uses. Solid = calls build steps (Director → CarsBuilder).
 Cars is immutable: private final fields, no public setters.

Figure 1. UML class diagram of the Custom Car Builder system.

3. Clean Code Principles Applied

This section highlights five Clean Code principles that were deliberately applied while implementing the pattern, each illustrated with an excerpt taken directly from the project's source code.

3.1 Meaningful, Intention-Revealing Names

Every class, method, and field name states exactly what it represents or does, so the code reads like the domain it models. Builder methods are named after the attribute they set (setColor, setWheels), the Director's methods read like a sentence describing the configuration being built (makeSportsCar, makeFamilyCar), and there is no abbreviation or ambiguous single-letter naming anywhere in the public API.

```
public interface CarsBuilder {
    CarsBuilder setColor(String color);
```

```

CarsBuilder setWheels(String wheels);
CarsBuilder setTransmission(String transmission);
CarsBuilder setEngine(String engine);
}

```

3.2 Single Responsibility Principle (SRP)

Each class has exactly one reason to change. `CarsObjectBuilder` is only responsible for assembling a `Cars` object; `CarsSpecSheetBuilder` is only responsible for formatting a text spec sheet; `CarsDirector` is only responsible for knowing which values make a sports car or a family car — it has no knowledge of how a car is represented. This separation is what allows two completely different outputs to be produced from one shared construction script.

```

public void makeSportsCar(CarsBuilder b) {
    b.setColor("Black").setWheels("Steel")
      .setTransmission("Manual").setEngine("V12");
}

```

3.3 Small Functions & a Fluent Interface

Every setter method does exactly one thing — assign one field — and returns this, which keeps each method to a single line and enables the fluent, chainable API used by the Director and the client. Concrete builders override the return type covariantly (returning their own type instead of the interface), which lets callers keep chaining without casting.

```

public CarsObjectBuilder setWheels(String wheels) {
    this.wheels = wheels;
    return this;
}

```

3.4 No Magic Numbers or Strings Scattered in Client Code

Literal values such as "Black", "Steel", or "V12" exist in exactly one place — inside the Director's `makeSportsCar` / `makeFamilyCar` methods. The client (Main) never types these values directly; it simply asks the Director for a named configuration. If a family car's default engine ever changes, only one line in `CarsDirector` needs to be edited.

```

public void makeFamilyCar(CarsBuilder b) {
    b.setColor("White").setWheels("Steel")
      .setTransmission("Automatic").setEngine("V4");
}

```

3.5 Immutability of the Product

`Cars` is declared with private final fields, exposes no public setters, and its constructor is package-private so that a `Cars` instance can only be created through a builder in the same package. Once built, a car's specification cannot be mutated from outside, which removes an entire class of bugs related to shared, changing state.

```

public class Cars {
    private final String color, wheels, transmission, engine;

    Cars(String color, String wheels,
         String transmission, String engine) {

```

```
        this.color = color;
        this.wheels = wheels;
        this.transmission = transmission;
        this.engine = engine;
    }
}
```

3.6 Fail Fast with a Clear, Descriptive Exception (bonus)

CarsObjectBuilder validates its internal state before producing a result, throwing an `IllegalStateException` if any required field was never set. This means an incomplete car can never be constructed, and failures are reported immediately at the point of misuse rather than silently producing a broken object.

```
public Cars getResult() {
    if (color == null || wheels == null ||
        transmission == null || engine == null) {
        throw new IllegalStateException(
            "All car fields are required");
    }
    return new Cars(color, wheels, transmission, engine);
}
```

4. Conclusion

This assignment implemented the Builder design pattern to construct a car from a single, reusable sequence of steps, while allowing that sequence to yield two distinct outputs: an immutable domain object and a formatted text spec sheet. `CarsDirector` demonstrates the core value of the pattern — it depends only on the abstract `CarsBuilder` interface, so the exact same `makeSportsCar` / `makeFamilyCar` logic can drive either builder without modification. Combined with an immutable product, fail-fast validation, and a fluent, self-documenting API, the implementation satisfies the assignment's structural requirements while remaining easy to extend — for example, adding a new trim level only requires a new Director method, and adding a third representation (e.g., a JSON export) only requires a new builder class that implements the existing interface.

5. GitHub Repository

The full source code, commit history, and README are available at:
<https://github.com/krein63/Assignment1-CarBuilder>